Compliant Manager

Aplikacja do zarządzania reklamacjami

Kierunek: Informatyka

Członkowie zespołu:
Jakub Trembaczowski
Tymon Bujak
Daniel Kocot

Gliwice, 2025

Spis treści

1 Informacje ogólne **2**
1.1 Cel projektu 2
1.2 Zakres funkcjonalny 2
1.3 Grupa docelowa 2

2 Zespół projektowy **2**
2.1 Skład zespołu 2
2.2 Role i odpowiedzialności 2

3 Instalacja i konfiguracja **3**
3.1 Instalacja i konfiguracja środowiska 3
3.2 Uruchamianie aplikacji 3

4 Architektura systemu **3**
4.1 Stos technologiczny 3
4.2 Baza danych 3

5 Użycie i funkcjonalności **4**
5.1 Scenariusze użytkownika 4
5.2 Przykładowa funkcjonalność – zarządzanie zgłoszeniami . . 4
5.3 Przykładowe ekrany 6

6 API **10**

7 Testowanie **11**
7.1 Testy manualne 11
7.2 Wizualizacja napotkanego problemu 12

8 Wnioski i rekomendacje **14**

9 Bibliografia / źródła **14**

1 Informacje ogólne

1.1 Cel projektu

Celem projektu jest stworzenie aplikacji „Compliant Manager”, której zadaniem jest usprawnienie procesu reklamacji i zwrotów oraz zapewnienie efektywnego przechowywania informacji o bieżących i archiwalnych zgłoszeniach.

1.2 Zakres funkcjonalny

Aplikacja obejmuje:

- obsługę zgłoszeń reklamacyjnych (dodawanie, edycja, przypisywanie, zmiana statusów),
- automatyczne informowanie klientów o zmianach statusu sprawy,
- możliwość korzystania z aplikacji przez interfejs webowy oraz za pośrednictwem API.

1.3 Grupa docelowa

Docelowi użytkownicy aplikacji to wewnętrzni pracownicy firm prowadzących sprzedaż produktów lub świadczących usługi podlegające reklamacji lub zwrotom. Aplikacja jest generyczną bazą, która może być dostosowana do indywidualnych wymagań klienta.

2 Zespół projektowy

2.1 Skład zespołu

1. Jakub Trembaczowski
2. Tymon Bujak
3. Daniel Kocot

2.2 Role i odpowiedzialności

Wszyscy członkowie zespołu byli zaangażowani w fazę projektową aplikacji.

- Jakub Trembaczowski: prace w części bazy danych, jej implementacji w kodzie i projektu backendowego aplikacji
- Tymon Bujak: prace w części bazy danych, implementacji m.in. serwisów aplikacji w projekcie backendowym aplikacji, oraz utworzenie i konfiguracja usług Azure
- Daniel Kocot: prace w części klienckiej aplikacji, łączenie z API, zabezpieczenia i implementacja identity framework

3 Instalacja i konfiguracja

3.1 Instalacja i konfiguracja środowiska

Aplikacja „Compliant Manager” wymaga środowiska opartego na platformie .NET oraz skonfigurowanych usług Microsoft Azure. Przed uruchomieniem należy:

- Zainstalować .NET SDK (w wersji zgodnej z ASP.NET Core).
- Zapewnić skonfigurowane usługi Azure

3.2 Uruchamianie aplikacji

1. Uruchomienie backendu:
   - Wejść do katalogu projektu API.
   - Skonfigurować dane połączenia do bazy i zasobów Azure z których zostanie pobrana reszta konfiguracji.
   - Uruchomić aplikację lokalnie lub hostować na serwerze.
2. Uruchomienie aplikacji klienckiej:
   - Wejść do katalogu projektu Blazor.
   - Zapewnić połączenie z backendem.
   - Uruchomić aplikację lokalnie lub hostować na serwerze.

Po poprawnym wdrożeniu użytkownicy uzyskują dostęp do aplikacji webowej oraz mogą komunikować się z API.

4 Architektura systemu

4.1 Stos technologiczny

Aplikacja „Compliant Manager” została zaprojektowana w architekturze MVC przy użyciu następujących technologii:

- Backend: ASP.NET Core z wykorzystaniem Entity Framework Core do komunikacji z bazą danych.
- Frontend: Webowa aplikacja napisana w technologii Blazor, zapewniająca pełną integrację z API.
- Bezpieczeństwo: Mechanizmy autentykacji i autoryzacji użytkowników.
- Środowisko chmurowe: Microsoft Azure jako platforma hostingu i przechowywania konfiguracji.

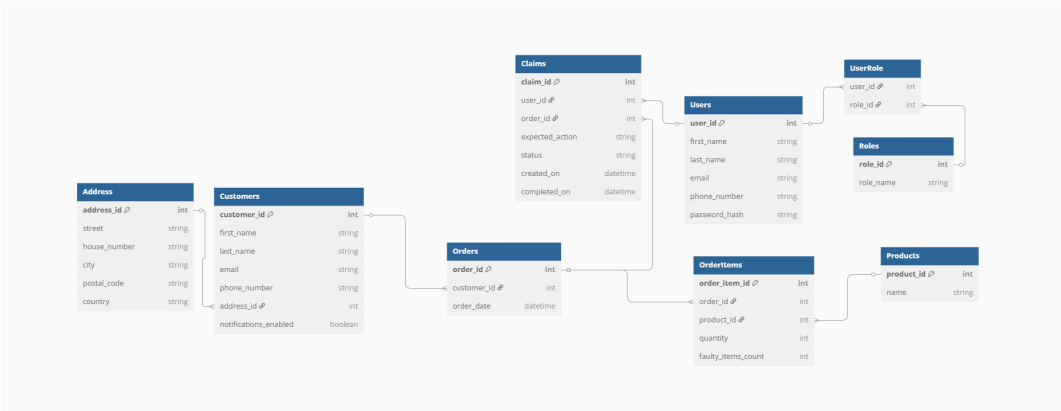
4.2 Baza danych

System wykorzystuje relacyjną bazę danych kompatybilną z Microsoft SQL Server. Kluczowe elementy przechowywane w bazie obejmują:

- dane zgłoszeń reklamacyjnych i zwrotów,

- powiązane informacje o klientach i produktach (w minimalnym zakresie),
- dane użytkowników systemu (konsultantów).

Komunikacja z bazą danych odbywa się za pośrednictwem ORM Entity Framework Core, co zapewnia elastyczność i bezpieczeństwo danych.



Panel Klientów

5 Użycie i funkcjonalności

5.1 Scenariusze użytkownika

- Zgłoszenie reklamacji przez klienta: Klient inicjuje proces reklamacyjny, korzystając z zewnętrznej aplikacji klienckiej lub kontaktując się z firmą. Zgłoszenie zostaje zarejestrowane w systemie — automatycznie przez API lub ręcznie przez konsultanta. Następnie przypisywane jest do konkretnego pracownika, który rozpatruje sprawę. Po podjęciu decyzji klient otrzymuje odpowiednią informację mailową wraz z dalszymi instrukcjami. Proces kończy się zwrotem środków lub dostarczeniem naprawionego/nowego produktu.

5.2 Przykładowa funkcjonalność – zarządzanie zgłoszeniami

Backend aplikacji odpowiada za obsługę procesów biznesowych związanych ze zgłoszeniami reklamacyjnymi oraz użytkownikami systemu. Warstwa kontrolerów (`Controllers`) komunikuje się z logiką biznesową zawartą w `Services/Implementations` i operuje na danych z bazy (`Data/Entities`), używając repozytoriów (`Repositories`).

1. Kluczowe komponenty:
   - `ClaimService.cs` – odpowiada za tworzenie, edytowanie i zarządzanie statusem zgłoszeń reklamacyjnych. Umożliwia przypisywanie spraw

do konsultantów i przekazuje informacje do warstwy wysyłki powiadomień.

- `CustomerService.cs` – zarządza informacjami o klientach przypisanych do zgłoszeń, umożliwiając przypięcie danych kontaktowych, np. w celu wysyłki wiadomości mailowej.
- `MailingService.cs` – obsługuje automatyczne generowanie i wysyłkę powiadomień e-mail do klientów w odpowiedzi na zmiany w statusie zgłoszeń, bazując na zdefiniowanych szablonach.
- `UserService.cs` – odpowiada za operacje na użytkownikach systemu, tj. tworzenie kont, pobieranie listy konsultantów i edycja.

2. Tożsamość i uwierzytelnianie użytkowników – Identity

Aplikacja wykorzystuje ASP.NET Core Identity do zarządzania użytkownikami i rolami. Rozszerzone klasy `ApplicationUser` i `ApplicationRole` definiują strukturę tożsamości:

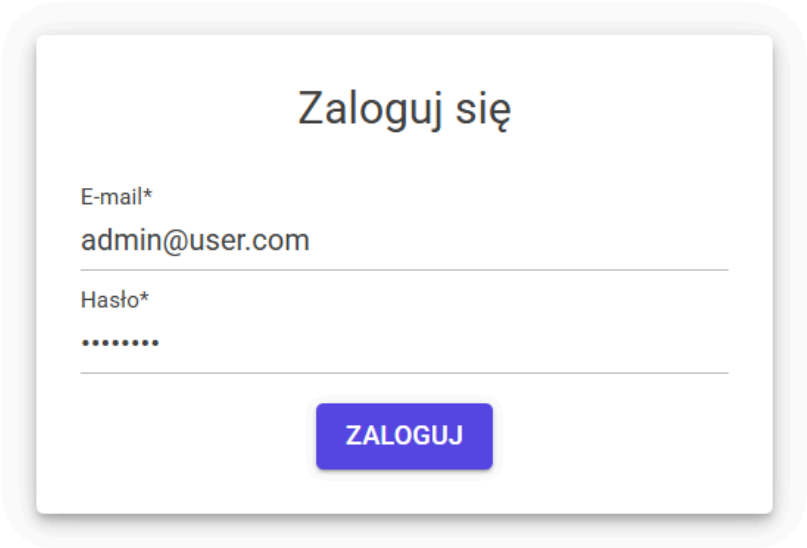
- `ApplicationUser.cs` – przechowuje dane zalogowanych użytkowników, w tym nazwę, adres e-mail oraz przypisane role.
- `ApplicationRole.cs` – umożliwia różnicowanie uprawnień między grupami użytkowników (np. konsultant, administrator).
- Serwisy integrujące Identity (`UserService.cs`, `AuthService.cs`) odpowiadają za:
  - rejestrację i logowanie użytkowników,
  - generowanie tokenów JWT do autentykacji,
  - weryfikację ról w oparciu o IdentityRole i wymagania systemowe.

Dzięki zastosowaniu Identity oraz JWT, każda akcja na danych zgłoszeń jest wykonywana w kontekście uwierzytelnionego użytkownika, co zapewnia bezpieczeństwo i kontrolę dostępu.

Logika systemu została rozdzielona na warstwy zgodnie z zasadami czystej architektury, co zapewnia przejrzystość kodu i łatwość utrzymania. Dzięki integracji z usługami Azure, przechowywanie danych wrażliwych i konfiguracja odbywają się bezpiecznie i centralnie.

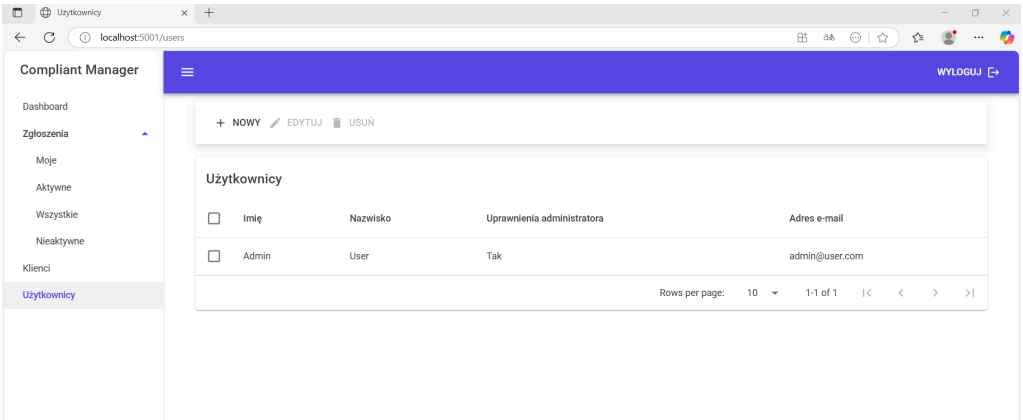
5.3 Przykładowe ekrany

1. Panel logowania

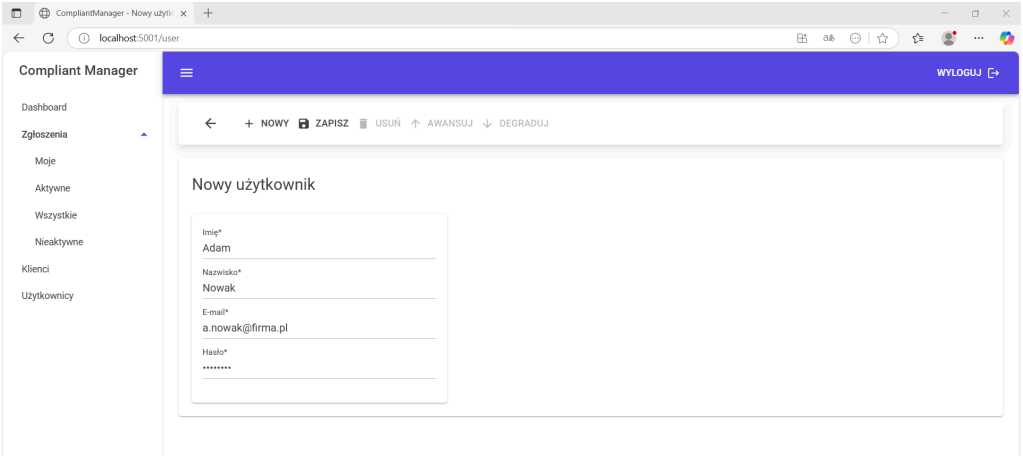


Panel Logowania

2. Zarządzanie Użytkownikami

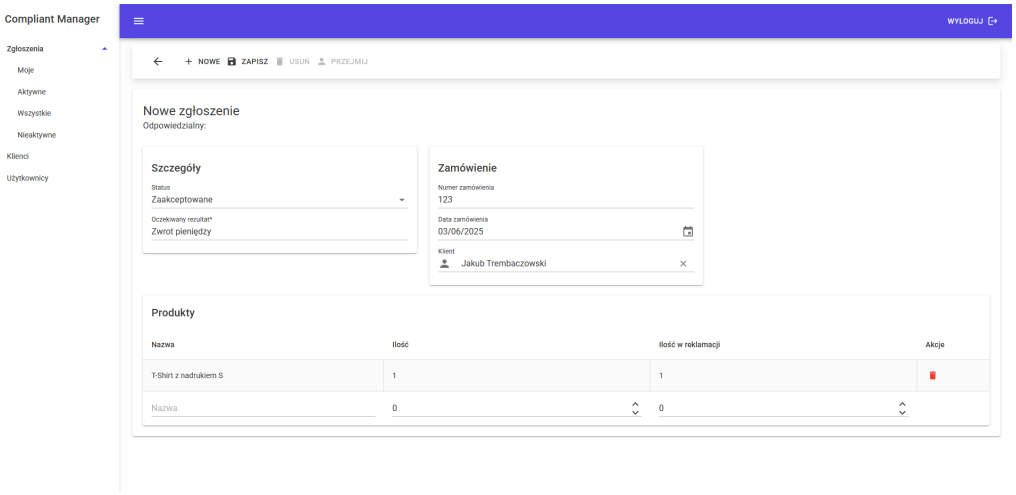


Panel Użytkowników

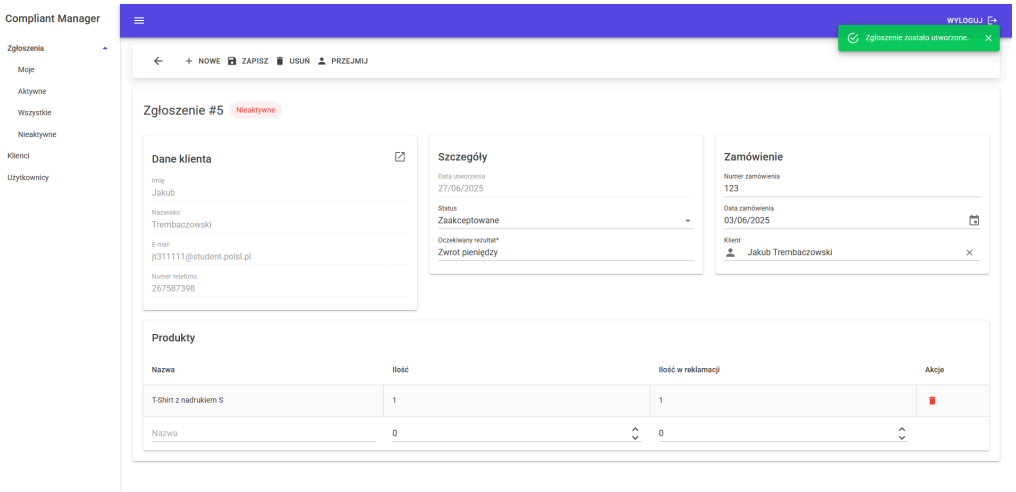


Panel Użytkowników - dodawanie użytkownika

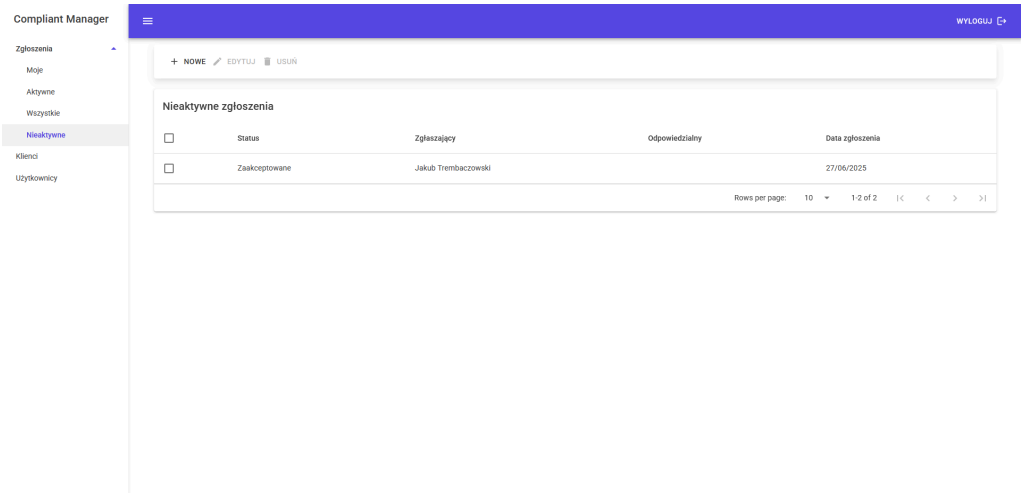
3. Zarządzanie klientami



Panel Zgłoszeń - dodawanie

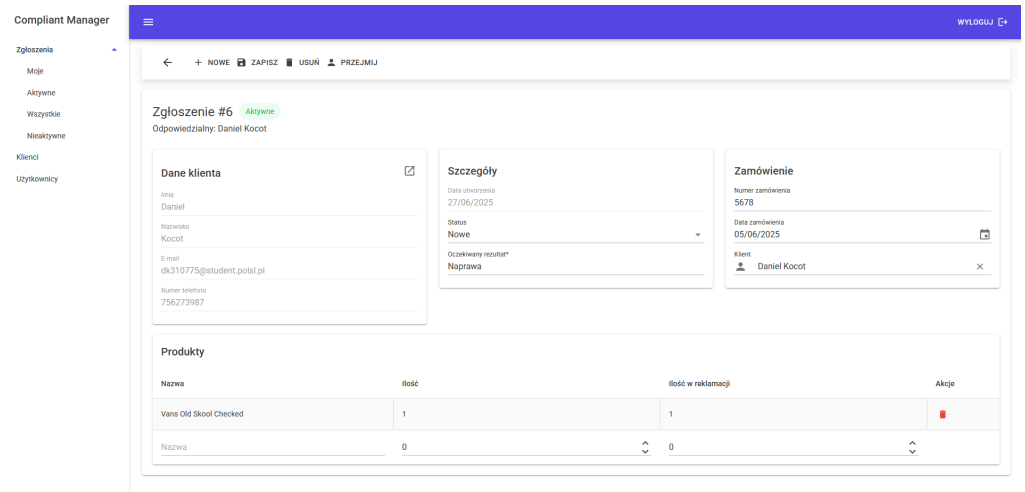


Panel Zgłoszeń - edycja

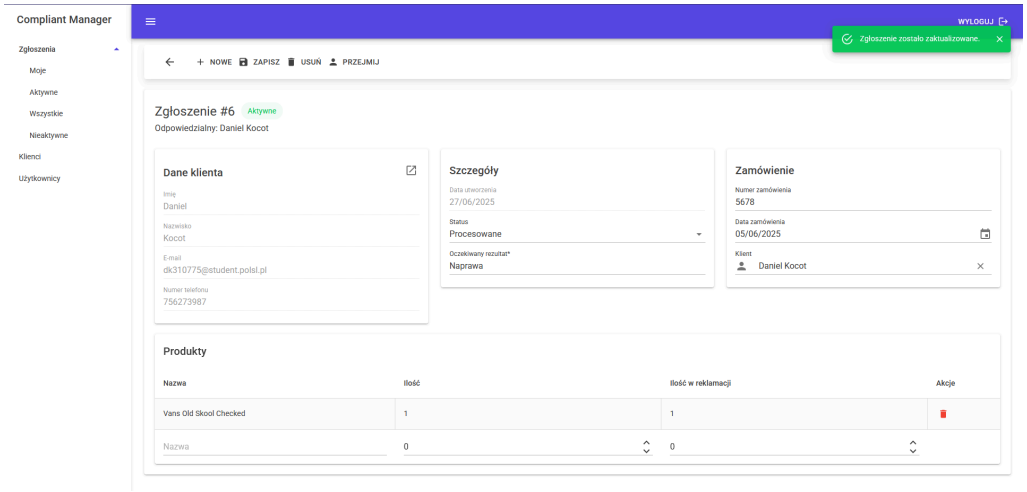


Panel Zgłoszeń - nieaktywne

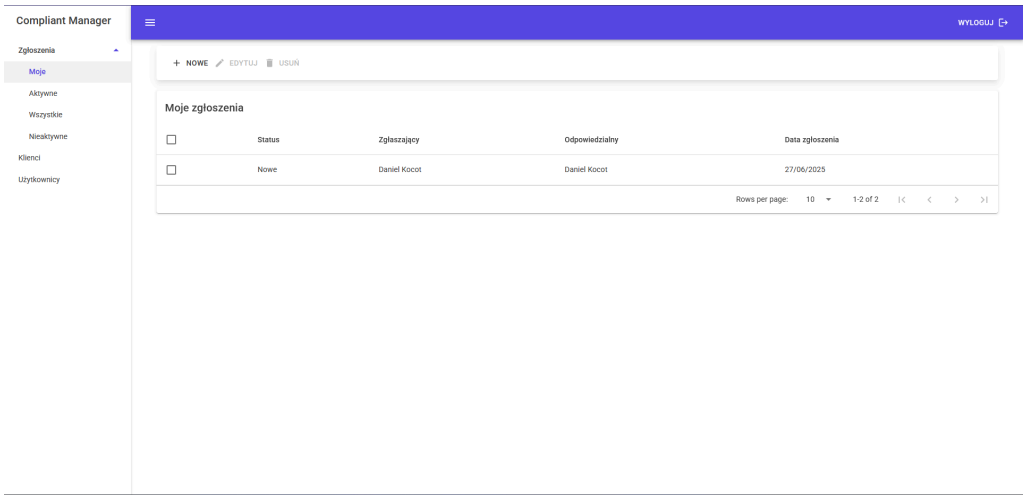
4. Zarządzanie zgłoszeniami



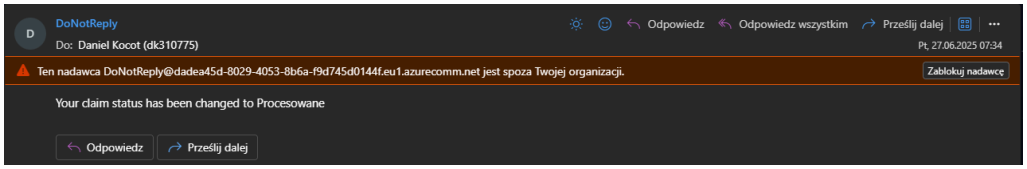
Panel Zgłoszeń - dodawanie zgłoszenia aktywnego



Panel Zgłoszeń - zgłoszenie z przypisanym konsultantem



Panel Zgłoszeń - moje



Mail o zmianie statusu

6 API

Auth

POST /api/Auth/register

POST /api/Auth/login

GET /api/Auth/me

GET /api/Auth/exists

Endpointy autentykacji

User

GET /api/User

PATCH /api/User

GET /api/User/{id}

DELETE /api/User/{id}

GET /api/User/count

POST /api/User/deleteMultiple

Endpointy użytkowników

Customer

GET /api/Customer

POST /api/Customer

PATCH /api/Customer

GET /api/Customer/{id}

DELETE /api/Customer/{id}

GET /api/Customer/count

POST /api/Customer/deleteMultiple

Endpointy klientów

Claim

GET /api/Claim

POST /api/Claim

PATCH /api/Claim

GET /api/Claim/{id}

DELETE /api/Claim/{id}

GET /api/Claim/count

POST /api/Claim/deleteMultiple

Endpointy zgłoszeń

7 Testowanie

7.1 Testy manualne

Aplikacja została przetestowana pod kątem poprawności działania oraz potencjalnych błędów. Wersja finalna nie wykazuje błędów krytycznych i działa w sposób stabilny zarówno po stronie klienta, jak i serwera. Przeprowadzone testy obejmowały m.in. weryfikacje:

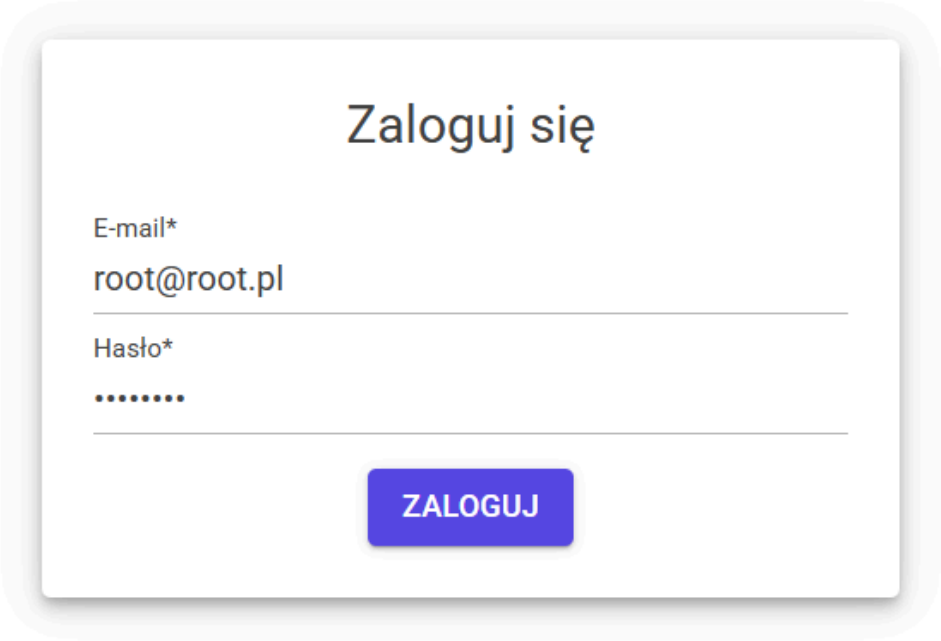
- logiki biznesowej,
- uprawnień użytkowników,
- działania formularzy,
- komunikacje z API,

Najpoważniejszym zidentyfikowanym problemem była możliwość wykonywania operacji przez użytkownika, którego konto zostało usunięte. Sytuacja występowała w przypadku, gdy użytkownik był wcześniej poprawnie zalogowany, a jego konto zostało usunięte w trakcie aktywnej sesji. Pomimo usunięcia konta z systemu, użytkownik mógł wciąż wykonywać działania dzięki zachowanemu w pamięci przeglądarki tokenowi.

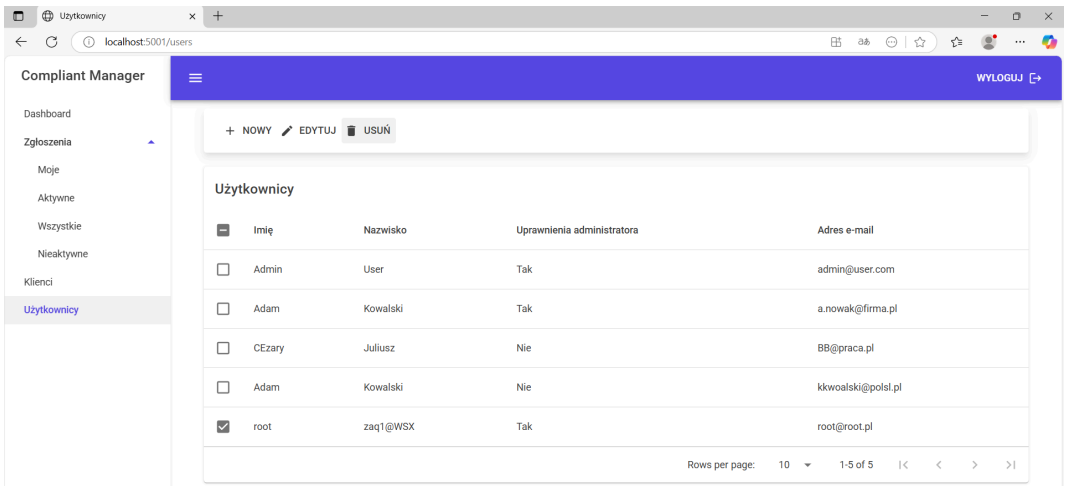
Problem wynikał z braku natychmiastowej weryfikacji stanu konta po stronie serwera przy każdym żądaniu. Został on rozwiązany poprzez modyfikację logiki autoryzacyjnej – wprowadzono dodatkowe sprawdzenie, czy użytkownik nadal istnieje w bazie danych, co uniemożliwia dalsze korzystanie z aplikacji po usunięciu konta.

Po wdrożeniu poprawki aplikacja została ponownie przetestowana i nie stwierdzono dalszych nieprawidłowości w tym zakresie.

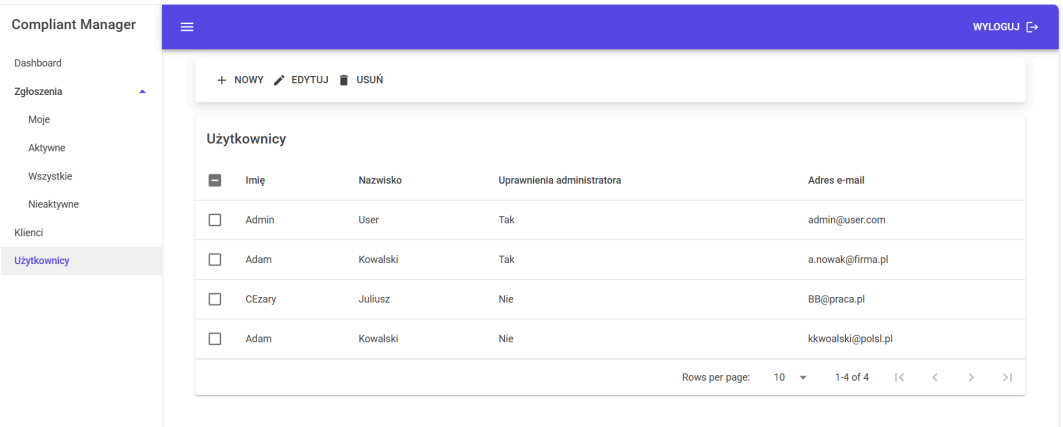
7.2 Wizualizacja napotkanego problemu



Logowanie na (jeszcze istniejącego) użytkownika

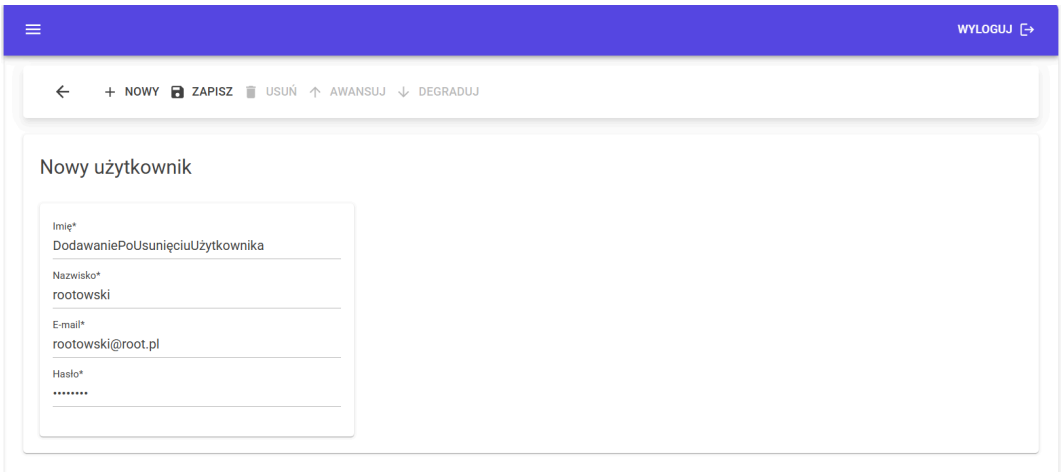


Usunięcie użytkownika root



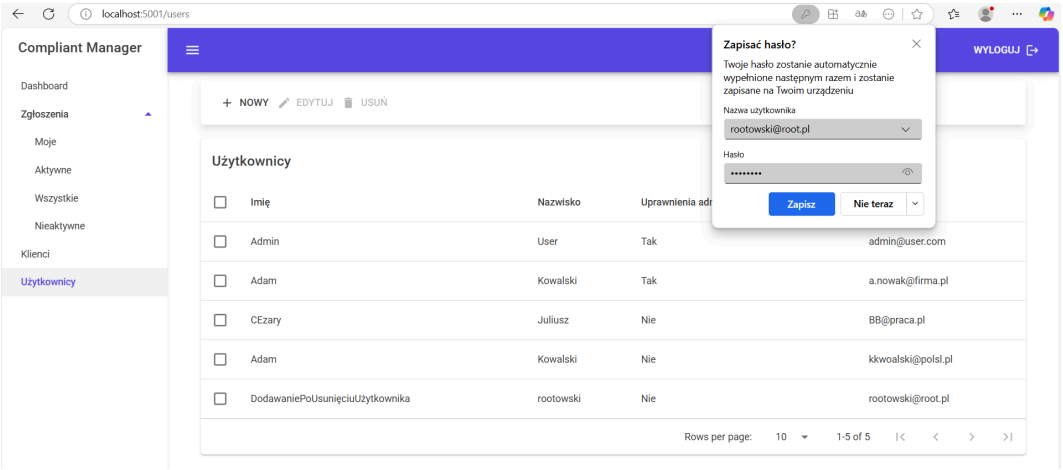
Panel Użytkowników

(Brak w bazie użytkownika root)



Panel Użytkowników - dodawanie użytkownika

(przez usuniętego użytkownika root)



Panel Użytkowników - dodanie użytkownika

8 Wnioski i rekomendacje

Projekt aplikacji „Compliant Manager” został zrealizowany zgodnie z przyjętymi założeniami. W trakcie realizacji nie wprowadzano żadnych istotnych zmian w zakresie funkcjonalnym ani technologicznym. Wszystkie planowane moduły zostały zaimplementowane i przetestowane.

Aplikacja spełnia wymagania dotyczące obsługi zgłoszeń reklamacyjnych, umożliwiając ich rejestrację, edycję, przypisywanie do konsultantów oraz automatyczne informowanie klientów o zmianach statusu sprawy. Zastosowanie platformy Azure oraz technologii ASP.NET Core i Blazor pozwoliło na stworzenie obiektowej aplikacji.

Dalsza rozbudowa aplikacji może obejmować integrację z systemami zamówień realizowanych poprzez strony internetowe. Pozwoliłoby to na automatyczne pobieranie danych o zakupach klientów, co usprawniłoby proces weryfikacji reklamacji i zwiększyło automatyzację pracy konsultantów, a także poprzez podłączenie do systemów przesyłek kurierskich, umożliwiające m.in. generowanie etykiet nadawczych, śledzenie statusów przesyłek oraz automatyzację procesu zwrotu towarów. Dodatkowo, w aplikacji można wprowadzać dalsze usprawnienia interfejsu graficznego i poprawiać ewentualne błędy.

9 Bibliografia / źródła

https://learn.microsoft.com/pl-pl/ef/core/

https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-9.0&tabs=visual-studio

https://learn.microsoft.com/en-us/samples/azure/azure-sdk-for-net/app--secrets-configuration/?tabs=visualstudio

https://learn.microsoft.com/en-us/azure/communication-services/quickstarts/email/send-email?tabs=windows%2Cconnection-string%2Csend-email-and-get-status-async%2Csync-client&pivots=programming--language-csharp